

Helix OTA — Production System + “Configured Cluster” Research (2026-06-23)

Revision: 1 **Last modified:** 2026-06-23T12:55:00Z **Method:** real repo inspection, FACT-with-file-citation or UNCONFIRMED (§11.4.6). NO commit by the research stream.

1. The production-ready system + real deps (FACT)

The system is the Go control plane **ota-server** (Gin modular monolith, `server/cmd/ota-server/main.go`, module `github.com/HelixDevelopment/helix_ota/server`). `server/internal/`: `api/` (REST — `artifact/device/deployment/rollout/release/telemetry/recall/audit/auth/health` + embedded manager `UI manager-dist/`), `store/`, `rollout/`, `device/`, `deviceemu/`, `fabric/`, `health/`, `transport/`, `config/`. Composes owned submodules `ota-protocol`, `ota-artifact-validator` (imported in `store/postgres.go`), `ota-rollout-engine`, `ota-telemetry-schema`, `http3`, `ota-update-engine-bridge/ota-android-agent`.

Real prod deps (`config/config.go` + `server/Dockerfile`): **PostgreSQL** (`HELIX_DATABASE_URL`), **ed25519 artifact pubkey** (`HELIX_ARTIFACT_PUBKEY`), `token secret`, `admin creds`, `HELIX_ARTIFACT_BASE_URL`, `TLS/HTTP3`. Trust boundary **CONFIRMED**: `handlers_artifact.go:resolvePublicKey` takes the key **ONLY** from server config, never the request.

2. FACT-correction: the pgx Repository IS implemented

`server/internal/store/postgres.go` (31 KB) + `postgres_fabric.go`: real `jackc/pgx/v5+pgxpool`, `var _ Repository = (*PostgresRepository)(nil)`, `NewPostgresRepository` opens+pings a pool, `Migrate()` applies embedded `schema_postgres.sql`, NO `TODO/FIXME/panic`. A shared `contract_test.go` asserts `memory[?]postgres` parity. Wiring (`main.go:41–67`): `HELIX_DATABASE_URL` set $\Rightarrow$ `pg repo` + `rollout.NewPostgresStore`; unset $\Rightarrow$ in-memory. **Real-DB mode already exists and is one env var away.** (The coverage audit’s understated `store/rollout` numbers are because the `-tags` integration `pgx` tests weren’t run, NOT missing code.)

3. Does “our configured cluster” exist? — ASPIRATIONAL for OTA (FACT)

No k8s/Swarm/Nomad/multi-node config anywhere. What EXISTS (`.env.example` + `scripts/distribute_stack.sh`): a **single-host rootless-podman remote-distribution capability** — `thinker.local` (LIVE: SSH key + rootless podman 4.9.3 + podman-compose, §11.4.161), `amber.local` (SSH ok, podman NOT installed ⇒ honest SKIP), `nezha.local` (Android-emulator/cuttlefish host, not a distribution target). **But `distribute_stack.sh` deploys the HelixTrack tracker stack (`deploy/helixtrack/compose.helixtrack.yml`), NOT `ota-server`.** Verdict: the “cluster” = one reachable podman host (`thinker`) and the OTA system is not deployed there. **Aspirational for OTA.**

4. On-demand boot — wired vs missing

ALREADY WIRED (§11.4.76): real PostgreSQL booted on-demand via the containers brick `pkg/{boot,compose,health,runtime}` in `server/internal/store/postgres_integration_test.go` (build tag `integration`, `server/deploy/postgres.compose.yml` port 55432; rollout port 55445; §11.4.119 lock) — `cd server && go test -tags integration ./internal/store/ ./internal/rollout/`. Real server-container + emulator pod via `tests/emulator/tier1_container_e2e.sh` (BUT runs the server with the **in-memory** store).

MISSING (the plan): **no combined `ota-server`+`postgres` compose stack** for one-command real-system (HTTP+DB) boot. Concrete plan: 1. Author `server/deploy/system.compose.yml` = `postgres` + `ota-control-plane` (`HELIX_DATABASE_URL=postgres://helix:helix@postgres:5432/helix_ota` + `pubkey/secret/admin` envs + `/readyz` healthcheck). 2. Add a Go boot harness (mirror `postgres_integration_test.go`) that boots the COMBINED stack via `pkg/{boot,compose,health}` (rootless §11.4.161), waits `/readyz`, returns the base URL — single entry point for `integration/e2e/full-auto/stress/security/chaos` (§11.4.27 real-system). 3. For a real “cluster” run: add an OTA-system compose under `deploy/` and have `distribute_stack.sh` deploy it to `thinker`, then run suites against `https://thinker.local:<port>`.

Foundational gaps (§11.4.6)

- **G1 (primary blocker):** no combined `server`+`postgres` compose ⇒ no one-command real-system boot; `tier1 e2e` is in-memory only.
- **G2:** “configured cluster” aspirational for OTA — `thinker` runs HelixTrack, not `ota-server`; no multi-node OTA cluster design.
- **G3:** `amber.local` not podman-ready (only `thinker` is live).
- **G4:** real-DB tests are `-tags integration` opt-in needing a podman/docker runtime; default `go test ./...` is in-memory.

NOT gaps (corrected): `pgx` Repository is implemented + contract-tested; server Dockerfile exists. Real-DB / real-system testing is blocked only by the missing combined boot stack (G1), not by missing persistence code.